

Fiche Projet – Trombinoscope connecté (Dev + Docker + CI/CD)

1) Contexte

L'école veut centraliser les **élèves**, leurs **classes** et **photos**, puis générer des **trombinoscopes** (par classe, promo, filière) en PDF/HTML. L'app doit être **conteneurisée** et **déployée via CI/CD**.

2) Fonctionnel (ce que l'app doit faire)

- CRUD **Élèves** (nom, prénom, email, classe, photo).
- CRUD **Classes** (ex. 3A, 3B, M1 MIAGE, etc.).
- Import **CSV** d'élèves (création en masse).
- Upload **photo** par élève (JPEG/PNG, < 5 MB).
- Génération d'un **trombinoscope**:
 - **HTML** responsive (grille de vignettes + noms).
 - **PDF** téléchargeable (A4 portrait, 30–40 photos/page, marges).
- Recherche/filtre: par **classe**, **promo**, **nom**.
- Rôles (option) : **admin** (tout), **enseignant** (lecture + génération).
- Journalisation minimale: qui a généré quel PDF, quand.

Note RGPD/consentement (texte fixe dans le footer des exports).

3) Modèle de données (SQL)

Tables minimales (ex. Postgres/SQLite) :

- classes(id, label, year, created_at)
- students(id, first_name, last_name, email, class_id FK, photo_url, created_at)
- users(id, email, password_hash, role [admin|teacher], created_at) (*option*)
- exports(id, class_id, format [html|pdf], path, created_by, created_at) (*optionnel mais utile*)

Index utiles : students(class_id), students(last_name)

4) API (REST)

- **Classes**
 - GET /api/classes – liste
 - POST /api/classes – créer
 - PUT /api/classes/:id – maj
 - DELETE /api/classes/:id – supprimer
- **Élèves**
 - GET /api/students?class_id=&q= – liste/filtre
 - POST /api/students – créer (JSON)
 - POST /api/students/import – **CSV** (multipart)
 - PUT /api/students/:id – maj
 - DELETE /api/students/:id – supprimer
 - POST /api/students/:id/photo – **upload** (multipart)
- **Trombinoscope**
 - GET /api/trombi?class_id=&format=html|pdf – génère & retourne URL
- **Auth (option)**
 - POST /api/auth/login – JWT
 - GET /api/me – profil

Formats CSV attendus (entêtes) :

first_name,last_name,email,class_label,year,photo_url?

5) Frontend (simple, suffisant)

- Page **Classes** : liste, création/édition/suppression.
- Page **Élèves** : grille + formulaire + upload photo + import CSV.
- Page **Générer** : choisir classe + format (HTML/PDF) → bouton **Générer** → lien de téléchargement.
- Feedback clair (toasts, erreurs de validation).

Tech au choix : **React/Vite** (SPA) **ou** server-rendered (Django/Symfony/Laravel/Flask + Jinja/Twig).
Gardez-le simple.

6) Photos & traitement

- À l'upload :
 - Vérifier type/poids.
 - **Redimensionner** une vignette 300×300 (carrée), qualité 80%.
 - Stockage:
 - local (/uploads) **ou**
 - S3/MinIO (env: S3_BUCKET, S3_ENDPOINT, S3_KEY, S3_SECRET).
- Option bonus : **crop automatique visage** (OpenCV/face-detection) si dispo.

7) Génération du PDF

- Approches possibles :
 - HTML → PDF via **wkhtmltopdf** / **WeasyPrint**.
 - Bibliothèque PDF (jsPDF/PDFKit) si back/Node.
- Mise en page : grille 5×6, logo établissement, titre “Trombinoscope – Classe XXX (Année)”, **footer RGPD**.

8) Tests à écrire (minimum)

- **Backend** :
 - Création classe/élève, import CSV (happy path + erreurs).
 - Upload photo → génère vignette (fichier existe).
 - Génération trombi HTML (statut 200) et PDF (fichier non vide).
- **Frontend** (si React) :
 - Form “nouvel élève” (validation).
 - Import CSV (afficher résumé : N créés / K en erreur).

9) Dockerisation

Dockerfile backend (ex. Node/Express ou Python/Flask) :

- Multi-stage (builder + runtime minimal).
- Variables PORT, DATABASE_URL, STORAGE (local/s3), JWT_SECRET.
- Expose PORT.

Dockerfile frontend :

- Build (Vite) → **Nginx** statique.

docker-compose.yml (dev) :

- web (backend), frontend, db (postgres), minio (option), wkhtmltopdf si requis.
- Volumes pour uploads/ et base.

10) CI/CD (GitHub Actions exemple)

Trois jobs : **test** → **build** → **deploy**

.github/workflows/ci.yml (extrait)

- **test** : installe deps, lance tests back (+ front si choisi).
- **build** :
 - build images Docker (backend, frontend)
 - push sur **registry** (Docker Hub ou GHCR).
- **deploy** :
 - sur main: SSH vers VM → docker compose pull && up -d
 - ou déploiement **Render/Fly.io/Dokku/CapRover**.

Pensez secrets CI : REGISTRY_USER, REGISTRY_TOKEN, SSH_HOST, SSH_KEY...

11) Environnements & config

- Fichier .env (dev) et **secrets CI** (prod).
- Variables clés :
 - DATABASE_URL
 - STORAGE=local|s3 (+ clés S3 si s3)
 - UPLOAD_DIR=/data/uploads
 - JWT_SECRET (si auth)
 - PDF_ENGINE=wkhtmltopdf|weasyprint

12) Résultat attendu (livrables)

- Repo Git avec :
 - /backend (API) + tests
 - /frontend (si SPA) + build
 - Dockerfile(s), docker-compose.yml
 - .github/workflows/ci.yml (ou .gitlab-ci.yml)
 - README.md clair (setup, env, commandes)
- Une **registry** contenant les images.
- Un **environnement déployé** accessible (URL demo) **ou** procédure de déploiement documentée.

13) Critères d'évaluation

- **Fonctionnel** : CRUD, import CSV, upload, génération HTML/PDF.
- **Qualité** : tests passent en CI, linter, README propre.
- **Ops** : Dockerfile(s) propres, images légères, pipeline CI/CD OK.
- **Déploiement** : script/procédure fiable; environnement démarre (docker compose up -d).
- **Bonus** : S3/MinIO, recadrage visage, rôles & auth, versioning d'exports, **healthcheck** & **/metrics** (Prometheus).

1) Contexte

Construire une **API de raccourcissement d'URL** qui génère des liens courts (ex. sho.rt/abc123), gère la **redirection** et collecte des **statistiques de clics** (date, referrer, user-agent, visiteurs uniques).

Objectif : une API claire, robuste, documentée et prête à être déployée.

2) Objectifs pédagogiques

- Concevoir et documenter une **API REST** (contrats, erreurs, statuts HTTP).
- Mettre en place une **persistance** propre (modèle de données, migrations).
- Implémenter un **module d'analytics** (événements + agrégations).
- Soigner les **aspects non-fonctionnels** : validation, sécurité basique, performance.
- (Option) **Conteneurisation + CI/CD + déploiement**.

3) Périmètre fonctionnel (MVP)

- Raccourcir une URL longue → obtention d'un **slug** et d'une **URL courte**.
- Redirection : accéder à l'URL courte renvoie vers l'URL cible.
- Statistiques par lien : **total, uniques, clics par jour, top referrers, top user-agents**.
- Recherche/liste des liens, **désactivation** d'un lien, **expiration** (option).

4) Règles & contraintes

- **Validation d'URL** (schéma http/https, longueur max).
- **Slug** unique (aléatoire Base62) + possibilité d'**alias personnalisé** s'il est libre.
- **Sécurité** :
 - protection contre **open redirect** (refuser javascript: etc.),
 - **rate-limit** pour la création de liens et l'accès aux stats,
 - ne **pas stocker d'IP en clair** (hash + sel) si vous traitez l'IP.
- **Performance** :
 - lecture de slug très fréquente → prévoir **cache**,
 - enregistrement des clics **sans ralentir la redirection** (asynchrone ou file).
- **Vie privée** : pas de données personnelles ; hashing pour l'unicité visiteurs.

5) Contrat d'API (sans code)

- **Créer un lien** : POST /api/links
Entrées : target_url, custom_slug?, expires_at?
Sorties : id, slug, short_url, target_url, created_at, expires_at, is_active
- **Redirection publique** : GET /:slug
Réponse : redirection (302 / 301) ou 410 si expiré/désactivé.
- **Détails d'un lien** : GET /api/links/:id
- **Stats d'un lien** : GET /api/links/:id/stats?range=7d|30d
Sorties : total_clicks, unique_visitors, by_day[], top_referrers[], top_user_agents[]
- **Administration rapide** :
 - PATCH /api/links/:id (ex. is_active:false)
 - DELETE /api/links/:id
- **Liste/Recherche** : GET /api/links?limit=&offset=&active=&search=
- **(Option) QR Code** : GET /api/links/:id/qrcode (image)
- **(Option) Lien protégé par mot de passe** : redirection conditionnelle

6) Étapes du projet (plan d'action)

Étape A — Cadrage & design

- Rédiger le **contrat d'API** (endpoints, schémas d'entrées/sorties).
- Dessiner le **modèle de données** et les règles (unicité du slug, validation d'URL).
- Lister les **codes d'erreur** (400, 401, 404, 409, 410, 422, 429, 500).

Étape B — MVP “fonctionnel”

- Implémenter la **création de lien** et la **redirection**.
- Ajouter la **liste/recherche** et le **détail** d'un lien.
- Mettre en place les **tests unitaires** sur ces fonctionnalités.

Étape C — Analytics

- Enregistrer un **événement de clic** à chaque redirection (avant le renvoi).
- Calculer **uniques** (par link_id, date, ip_hash).
- Exposer l'endpoint **stats** (agrégations par jour, top referrers/UA).
- Définir un **process d'agrégation** (tâche planifiée / job / vue matérialisée).

Étape D — Non-fonctionnels

- **Validation** stricte des entrées, messages d'erreur clairs.
- **Rate-limit** (création & stats), **désactivation** et **expiration**.
- **Cache slug** → **target** (invalidation en cas de changement).
- Journalisation et **observabilité** (logs structurés, healthcheck).

Étape E — Packaging & déploiement

- Rédiger un **README** (setup, variables d'environnement, commandes).
- Préparer la **documentation d'API** (OpenAPI/Swagger/Postman).
- (Option) **Dockerisation** + fichier de composition pour dev.
- (Option) **CI/CD** : tests → build image → déploiement.
- (Option) **Kubernetes** : Deployment, Service, Ingress, HPA (si demandé).

8) Plan de tests (à exécuter et montrer)

- **Création** : lien valide, collision d'alias (conflit), URL invalide (422).
- **Redirection** : 302 OK ; expiré (410) ; désactivé (410).
- **Analytics** : après N clics, les **stats** reflètent total/uniques/by_day.
- **Rate-limit** : dépassement → 429.
- **Sécurité** : refuser schémas non sûrs ; lien privé (si option) : accès avec/ sans mot de passe.
- **Performance** : latence de redirection stable (les clics n'aggravent pas le temps).

9) Livrables attendus

- **Dépôt Git** propre : code, migrations, scripts d'init.
- **Documentation d'API** (OpenAPI/Swagger ou collection Postman) et **README** complet.
- **Jeux de tests** (unitaires/intégration) avec résultats.
- **Démonstration** : appels d'API (création, redirection, stats) + quelques graphiques simples.

- (Option) **Image Docker** publiée, pipeline CI, URL de démo.

10) Extensions possibles (bonus)

- **QR codes** pour les liens.
- **Tags/projets** pour regrouper les liens d'une campagne.
- **Dashboard** minimal (graphes par jour).
- **Webhooks** (on-click) pour notifier un autre service.
- **A/B test** : alterner deux target_url et comparer les clics.

1) Contexte

Construire une **API d'images** qui :

- reçoit des images originales,
- génère **à la volée** (ou en “build”) des **miniatures** et **transformations** (resize, crop, format WebP/AVIF),
- **met en cache** les rendus,
- expose des **métriques** pour piloter des **tests de charge** (CPU-bound) et démontrer l'**autoscaling**.

2) Objectifs pédagogiques

- Concevoir une **API REST** claire pour la gestion des médias.
- Gérer **stockage, cache, headers HTTP** (ETag, Cache-Control), **sécurité** de traitement d'images.
- Définir des **SLO** et mener des **campagnes de tests de charge** (spike, stress, soak).
- (Option) Déploiement **Docker/Kubernetes + HPA** (autoscaling sur CPU/RAM).

3) Périmètre fonctionnel (MVP)

- **Upload** d'une image originale (JPEG/PNG/WebP), métadonnées basiques.
- **Transformation on-demand** par **URL signée** ou paramètres contrôlés :
 - resize (w, h, mode *fit/fill*), format (jpg/webp/avif), quality.
- **Cache** du rendu pour les requêtes suivantes (clé = hash de l'original + ops).
- **Récupération** de l'original et des rendus (avec ETag/Cache-Control).
- **Statistiques** : volumes, taux de hit du cache, latences p95/p99.
- **Observabilité** : /healthz, /metrics (Prometheus), logs structurés.

4) Contrat d'API (sans code)

Upload

POST /api/images (*multipart*)

- Entrées : fichier image (≤ 10 MB), filename?, content-type correct.
- Sorties : image_id, url_original, width, height, mime, created_at.
- Erreurs : 415 (type non supporté), 413 (trop gros), 422 (corrompu).

Rendu (on the fly, ciblé charge CPU)

GET /i/{image_id}?w=..&h=..&fit=cover|contain&fmt=jpg|webp|avif&q=1..95

- Réponse : binaire image + Content-Type, ETag, Cache-Control.
- Caching applicatif + HTTP (configurable).
- Erreurs : 400 (paramètres invalides), 404 (image inconnue), 429 (rate-limit).

Original

GET /o/{image_id} → renvoie l'original (pour comparaison perf/caching).

- GET /api/images/{image_id} → métadonnées.
- GET /api/stats → agrégats : nb uploads, rendus, hit-ratio, tailles, p95.
- DELETE /api/images/{image_id} (option).

Santé & métriques

- GET /healthz → ok.
- GET /metrics → exposition Prometheus (RPS, latences, cache hits, erreurs).

Option sécurité : URL **signées** pour les rendus (sig=...&exp=...) afin d'éviter l'explosion de variantes.

5) Stockage & cache (conceptuel)

- **Originals** : objet binaire (disque local en dev, **S3/MinIO** en prod).
- **Rendus** :
 - cache **en mémoire** (LRU) pour le hot path,
 - cache **persistant** (dossier/objet S3) pour hits inter-pods.
- **Clé de rendu** = sha256(image_id + params normalisés) → unique et stable.
- **Headers HTTP** : Cache-Control: public, max-age=..., ETag, Last-Modified.

6) Sécurité & robustesse

- **Validation stricte** des formats (refuser exotiques), **limites** de résolution (ex. max 8k) et de mémoire.
- **Protection image bomb** (ex. large dimension déclarée vs réelle), **timeout** de décodage.
- **Rate-limit** sur GET /i/* et POST /api/images.
- **Signatures** ou **liste blanche** de tailles autorisées (ex. $w \in \{200, 400, 800\}$) pour éviter un nombre infini de variantes.
- Journalisation **corrélée** (trace_id) et gestion **idempotente** des mêmes rendus.

7) SLO (exemples cibles à valider en charge)

- GET /i/{id}?w=400&fmt=webp : **p95 < 150 ms**, **erreurs < 1%** à **200 req/s** (avec cache chaud).
- Cache **hit-ratio** $\geq 85\%$ sur un trafic réaliste (5 tailles populaires).
- POST /api/images : **p95 < 300 ms** à **30 req/s**.
- GET /o/{id} (original) : **p95 < 120 ms** (CDN/objet store en lecture).

8) Scénarios de tests de charge (k6/JMeter/Locust)

1. Spike test (vague de rendus)

- 0 → 500 utilisateurs en 10 s, tous demandent **les 3 mêmes tailles** de 50 images populaires.
- Attendus : montée du **hit-ratio**, **latence p95** stable dès que le cache se réchauffe.

2. Stress test (variantes dispersées)

- Mélange : 70% GET /i (tailles variées), 20% GET /o, 10% POST /api/images.
- Ramping 50 → 600 vus sur 10 min.
- Objectif : observer le **point de rupture** (CPU saturé), comportement du **HPA**.

3. Soak test (endurance)

- Charge modérée constante (ex. 80 req/s) pendant 60–90 min.
- Vérifier **fuites mémoire**, **stabilité du cache**, erreurs cumulées.

4. Cache efficacy

- 95% des requêtes sur **5 tailles fixes** (400/800/1200, jpg/webp).
- Viser **hit-ratio** $\geq 90\%$, p95 significativement plus bas que “cold”.

5. Pathological payloads (robustesse)

- Images **très grandes** (8k), **corrompues**, **mauvais mime** → attendre 400/415/422 avec latence contrôlée, pas de crash.

9) Indicateurs à suivre (dashboards)

- **Techniques** : RPS, latences p50/p95/p99, taux d’erreurs par code, CPU/RAM/pods, latence I/O stockage, **hit-ratio cache** (mémoire + persistant).
- **Métier** : top tailles demandées, top formats, économies de bande passante grâce au cache (octets).
- **Autoscaling** : répliques HPA dans le temps vs CPU/mémoire.

10) Plan de travail

Étape A — Design & règles

- Spécifier **contrats d’API**, tailles autorisées, formats, stratégie de **cache** et **clés**.
- Définir **SLO & seuils** (thresholds) pour les tests.

Étape B — MVP fonctionnel

- Upload, rendu on-the-fly (1–2 formats), headers HTTP corrects, /healthz.
- Stockage local (dev) puis abstraction objet store.

Étape C — Cache & Observabilité

- Cache mémoire + persistant, invalidation basique.
- Exposer /metrics (compteurs, histogrammes), logs JSON.

Étape D — Charge & Résilience

- Rédiger 3 **scénarios de charge** (spike, stress, soak).
- Mettre en place **rate-limit**, tailles en **liste blanche**, timeouts.
- Rapports & recommandations (tailles “officielles”, TTL cache, ressources).

Étape E — (Option) K8s & HPA

- Déployer sur **Minikube/cluster** : Deployment, Service, Ingress.
- **Requests/Limits** ressources, **HPA** (CPU target 60–70%).
- Rejouer les tests, comparer avant/après autoscaling.

11) Jeux d’essai (dataset)

- 1 pack d’images **populaires** (portraits, paysages) 1–5 MB, diverses résolutions (1080p, 4k).
- 1 pack d’images **pathologiques** (grandes, corrompues, faux mimetypes).

- 5 **tailles officielles** pour les rendus (ex. 200, 400, 800, 1200, 1600).

12) Livrables attendus

- **Contrat d'API** (OpenAPI/Swagger ou collection Postman) + **README**.
- Scripts de **tests de charge** + rapports (latences, erreurs, hit-ratio).
- Dashboard (captures Grafana ou équivalent).
- (Option) Manifests **Kubernetes** + capture HPA en action.
- Note d'**architecture & choix** (cache, TTL, formats, sécurité).

13) Extensions possibles

- **File d'attente** de rendus lourds (worker asynchrone).
- **Préréglages** marketing (presets “card”, “cover”, “thumb”).
- **CDN** en façade + invalidation.
- **A/B** formats (JPEG vs WebP vs AVIF) avec comparaison de débit/latence.

Fiche Projet — Karaoké collaboratif (GitOps)

1) Contexte

Créer une petite plateforme de **karaoké en salle** : des **salons** (rooms) gérés par un hôte, une **file d'attente** de morceaux alimentée par les participants, des **paroles synchronisées** affichées à l'écran, et un **système de votes** (ex. "skip"). Le tout est livré et exploité via une démarche **GitOps** avec plusieurs environnements (**dev** / **staging** / **prod**).

2) Objectifs pédagogiques

- Travailler en **équipe** avec un vrai **flux Git** (branches, PR, revues).
- Mettre en place un **déploiement automatisé** déclenché par la gestion de versions.
- Gérer **plusieurs environnements** et des **configurations différenciées**.
- Préparer une **démonstration** claire et reproductible.

3) Résultat attendu (fonctionnalités minimales)

- Création/fermeture d'un **salon** avec un **code d'accès**.
- **Catalogue** de morceaux (titres, artistes) fourni par l'équipe.
- **File d'attente** : proposer un morceau, voir l'ordre, retirer/réordonner (par l'hôte).
- **Paroles** (fichiers fournis par l'équipe) affichées de façon **synchronisée**.
- **Votes** simples des participants (ex. "skip") influençant la session.
- **Page de salle** qui se **met à jour en direct** (sans recharger).
- **Tableau simple** de statistiques (ex. chansons jouées, tops).

4) Étapes à réaliser (plan d'action)

Étape A — Cadrage & préparation

1. Rédiger le **backlog** (liste des fonctionnalités) et les **user stories**.
2. Définir les **rôles** (hôte/participants) et les **règles** (ex. seuil de votes "skip").
3. Constituer un **jeu d'essai** : liste de morceaux (métadonnées) + quelques fichiers de paroles (synchronisés par vos soins).
4. Déterminer le **parcours de démo** (scénario public de 3–5 minutes).

Étape B — Maquettes & expérience utilisateur

1. Esquisser les **écrans** :
 - accueil / rejoindre un salon,
 - écran hôte (gestion de la file),
 - écran participants (proposer/voter),
 - écran d'affichage des **paroles**.
2. Définir les **messages d'état** (salon plein, morceau introuvable, vote pris en compte).

Étape C — Organisation GitOps

1. Mettre en place **deux dépôts** :
 - un dépôt **application** (code + doc utilisateur),
 - un dépôt **opérations** (manifeste de déploiement, configs d'environnements).
2. Créer les **trois environnements** : **dev** / **staging** / **prod** avec **configs distinctes** (par ex. tailles max de salon, règles de vote, messages).
3. Définir le **workflow d'équipe** : branches *feature*, **pull requests**, **revues** et **validation**.

4. Paramétrer un **déploiement automatique** sur fusion en branche cible (staging au minimum).

Étape D — Qualité & validation

1. Écrire des **tests de comportement** sur les parcours clés (créer un salon, ajouter un morceau, voter, afficher les paroles).
2. Définir des **vérifications de base** avant déploiement (checklist “prête à déployer”).
3. Préparer un **jeu de données** cohérent pour chaque environnement (ex. moins de morceaux en dev).

Étape E — Démonstration & restitution

1. Organiser une **session de démo** :
 - création d’un salon,
 - participants qui rejoignent,
 - ajout de 2–3 morceaux,
 - affichage des paroles,
 - vote “skip” et passage au suivant.
2. Présenter le **flux GitOps** : montrer une **modification de configuration** (ex. seuil de votes) appliquée automatiquement sur **staging**, puis publiée sur **prod**.
3. Remettre un **dossier de restitution** : guide utilisateur, guide d’exploitation (runbook), et retour d’expérience d’équipe.

5) Organisation & rôles (suggestion)

- **Owner produit** : tient le backlog, tranche sur le périmètre.
- **Référent qualité** : s’assure des tests, de la doc, de la cohérence.
- **Référent déploiement** : pilote la partie GitOps et les environnements.
- **Équipe dev** : prend les user stories, fait des PR courtes et fréquentes.

6) Livrables attendus

- **Backlog** priorisé + **maquettes** d’écrans.
- **Dépôt application** (code, doc utilisateur, jeux d’essai).
- **Dépôt opérations** (organisation des environnements, configurations).
- **Journal de bord** d’équipe (qui a fait quoi, points clés).
- **Guide de démo** (5 min) + **guide d’exploitation** (démarrer, arrêter, dépanner).

7) Critères d’évaluation

- **Fonctionnel** : salons, file d’attente, paroles visibles, votes qui comptent.
- **Collaboration** : branches, PR, revues, résolution de conflits.
- **GitOps** : déploiement automatique à la fusion, séparation claire **app / ops**, configs par environnement.
- **Qualité** : tests de parcours clés, documentation claire, démo fluide.
- **Professionalisme** : respect des engagements, gestion des priorités, restitution structurée.

8) Extensions (facultatives)

- **Scores** ou mini-jeu (ex. répondre à l’écran pour marquer des points).
- **Mode “soirée”** (file plus longue, seuil de votes différent, réglé par configuration).
- **Page “top titres”** et **historique des salons**.
- **Accessibilité** (contrastes, taille de police pour les paroles).

